

Scripts Execution

Overall Processing Summary

Mainly classifies into Batch & Streaming Processing

Batch Processing

- I. Historical Card Transactions Dataset has to be loaded from CSV file to HDFS. Further this data will be to hive Hbase tables.
- II. Member Score & card member data have been loaded to hive tables from RDS db by using sqoop import command.
- III. Card Lookup Table is generated with card Transactions and Member Data by applying transformation to get moving average and standard deviation of UCL.

Streaming Processing

- IV. Data arrives via Kafka Topics and post validating in kafka based on business rules marked as Fraud/Genuine and posted to card_transactions and card_lookup tables.

For the midterm submission we have completed 4 tasks given which mainly involved Batch processing framework.

Explanation of the solution to the batch layer problem is as follows.

Table Creation

We need to create all the necessary tables in Hbase & hive system for processing, this has been done by following below steps as a part of Task 1.

- **Task 1:** Load the transactions history data (card_transactions.csv) in a NoSQL database.

Note: Script loadNosql.sh and createHistorical.hql created to complete 1st task

- 1) Copy card_transactions.csv from your local system to EMR cluster using winscp application to /home/hadoop location
- 2) Once the file is available in EMR cluster, create hive directory in hbase system and perform hdfs copy operation to transfer file from local to hdfs location.
- 3) Once file has been loaded from local to HDFS system, create hql file to load the data into hive table for initial processing and then load it to Hbase table by creating proper hbase row keys and column structure.
- 4) Create **createHistorical.hql** file with all hql queries to load the data into hbase and hive table.

```
vi createHistorical.hql
```

```
CREATE EXTERNAL TABLE IF NOT EXISTS card_transactions_stg (  
  card_id STRING,  
  member_id STRING,  
  amount DOUBLE,  
  postcode STRING,  
  pos_id STRING,  
  transaction_dt STRING,  
  status STRING  
)  
ROW FORMAT DELIMITED  
FIELDS TERMINATED BY ','  
LOCATION '/user/hadoop/historical/card_transactions'  
tblproperties("skip.header.line.count"="1");  
  
LOAD DATA INPATH 'hdfs:/user/hadoop/hive/card_transactions.csv' OVERWRITE INTO  
TABLE card_transactions_stg;  
  
CREATE EXTERNAL TABLE IF NOT EXISTS card_transactions (row_key  
struct<card_id:string, pos_id:string, transaction_dt:string>, card_id STRING, pos_id  
STRING, transaction_dt STRING, member_id STRING, amount DOUBLE, postcode  
STRING, status STRING)
```

```

ROW FORMAT DELIMITED
COLLECTION ITEMS TERMINATED BY '~'
STORED BY 'org.apache.hadoop.hive.hbase.HBaseStorageHandler'
WITH SERDEPROPERTIES ('hbase.columns.mapping'='TD:card_id, TD:pos_id,
TD:transaction_dt,TD:member_id,TD:amount, TD:postcode, TD:status')
TBLPROPERTIES ("hbase.table.name" = "card_transactions");

Insert into card_transactions select
NAMED_STRUCT('card_id',card_id,'pos_id',pos_id,'transaction_dt', transaction_dt) as
row_key,card_id,pos_id,transaction_dt, member_id, amount, postcode, status from
card_transactions_stg;

CREATE VIEW IF NOT EXISTS last_ten_transactions
AS select card_id, member_id, amount, transaction_dt, postcode, rank() over (PARTITION
BY card_id ORDER BY unix_timestamp(transaction_dt , 'dd-MM-yyyy hh:mm:ss') desc,
amount desc) as ranking from card_transactions where status='GENUINE';

```

- 5) Create shell script to run hql file along with copy command for local to HDFS file transfer.
- 6) Run **sh loadNosql.sh** command to run the shell script for table creation and hdfs transfer.

```

vi loadNosql.sh

#!/bin/bash

hdfs dfs -rm -r hdfs:/user/hadoop/hive
hdfs dfs -mkdir hive
hdfs dfs -copyFromLocal /home/hadoop/card_transactions.csv
/user/hadoop/hive/card_transactions.csv

echo "create 'card_transactions', 'TD'" | sudo hbase shell -n
echo "create 'look_up_table', 'card_details', 'Member_details', 'Location', 'Rule_params'"
| sudo hbase shell -n

hive -f /home/hadoop/createHistorical.hql

```

- **Task 2:** Ingest the relevant data from AWS RDS to Hadoop.

-

Note: Script dataIngestion.sh created to complete the 2nd task.

Data related to card_member and member_score is stored in a central AWS RDS server. We need to use sqoop import command to pull the data from RDS db to our hdfs location.

- 1) Run below command to import card_member table from RDS to hdfs.

```
sqoop import --connect "jdbc:mysql://upgradawsrds1.cyaieic9bmnf.us-east-1.rds.amazonaws.com:3306/cred_financials_data?connectionCollation=latin1_swedish_ci" \  
--table card_member \  
--target-dir \  
/user/hadoop/sqoop/import/cred_financials_data/card_member \  
--username upgraduser \  
-m 1 \  
--password upgraduser
```

- 2) Run below command to import member_score table from RDS to hdfs.

```
sqoop import --connect "jdbc:mysql://upgradawsrds1.cyaieic9bmnf.us-east-1.rds.amazonaws.com:3306/cred_financials_data?connectionCollation=latin1_swedish_ci" \  
--table member_score \  
--target-dir \  
/user/hadoop/sqoop/import/cred_financials_data/member_score \  
--username upgraduser \  
-m 1 \  
--password upgraduser
```

- 3) After execution of above command, Data will be loaded in **/user/hadoop/sqoop/import/cred_financials_data** location.
- 4) Run below scripts with all above sqoop commands to ingest data into hdfs.

```
vi dataIngestion.sh  
sh dataIngestion.sh
```

- 5) Run below commands to verify files and count of data in each file.

```
hadoop fs -ls /user/hadoop/sqoop/import/cred_financials_data/  
hadoop fs -ls  
/user/hadoop/sqoop/import/cred_financials_data/card_member/  
hadoop dfs -ls sqoop/import/cred_financials_data/member_score  
  
hadoop fs -cat  
/user/hadoop/sqoop/import/cred_financials_data/card_member/part-m-  
00000 | wc -l  
  
hadoop fs -cat sqoop/import/cred_financials_data/member_score/part-m-  
00000 | wc -l
```

Task 3: Create a look-up table with columns specified earlier in the problem statement.

Note: Script **createLookup.sh** created to complete 3rd task

- 1) Create shell script **createLookup.sh** with command to create empty table with columns in in hbase system and run hql file to create same lookup table in hive and store the data into Hbase table.

```
#!/bin/bash  
  
echo "create 'look_up_table', 'card_details', 'Member_details', 'Location',  
'Rule_params'" | hbase shell -n  
  
hive -f /home/hadoop/loopuptable.hql
```

- 2) Create lookuptable.hql file with below query to generate hive external table which will be further store into hbase system.

```
CREATE EXTERNAL TABLE IF NOT EXISTS look_up_table (  
  card_id STRING,  
  card_purchase_dt STRING,  
  transaction_dt STRING,  
  member_id STRING,  
  member_joining_dt STRING,  
  country STRING,  
  city STRING,  
  UCL DOUBLE,  
  postcode STRING,  
  score INT  
)  
ROW FORMAT DELIMITED  
FIELDS TERMINATED BY ','  
STORED BY "org.apache.hadoop.hive.hbase.HBaseStorageHandler"  
WITH SERDEPROPERTIES ("hbase.columns.mapping" =  
"card_details:card_purchase_dt,  
card_details:transaction_dt,Member_details:member_id,  
Member_details:member_joining_dt,  
Location:country,Location:city,Rule_params:UCL,Rule_params:postcode,  
Rule_params:score")  
TBLPROPERTIES ("hbase.table.name" = "look_up_table");
```

- 3) Now run **sh createLookup.sh** command to generate table in both hive and hbase system.
- 4) Validate the table creation by using below command.

```
Hbase:  
  list  
  scan "look_up_table"  
Hive:  
  show tables;  
  select count(*) from look_up_table;
```

- **Task 4: After creating the table, you need to load the relevant data in the lookup table.**

Before loading data into lookup table in hbase we need to load the data from card_member and member_score files from hdfs system to hive tables.

We need to create card_member and member_score tables in hive system to load the data from files. Once the data has been loaded, we need to apply transformation to get Moving average and standard deviation of the last 10 transactions for each card_id.

Once we have all the data after transformation, we have to insert all the data into both hive and Hbase look_up_table.

Steps:

- 1) Create an external table card_member by using hive query language and store in hadoop location.
- 2) Create an external table member_score by using hive query language and store in hadoop location.
- 3) Load the data into hive tables from hadoop file system which contain extracted data from RDS db.
- 4) Insert the data into look_up_table table which was created in Hbase system from card_member and member_score table after applying join on tables to get moving average and standard deviation using hql.
- 5) Best way is to create PreAnalysis.hql file with below queries and run it by using hive -f PreAnalysis.hql or through shell script command.

```
CREATE EXTERNAL TABLE IF NOT EXISTS card_member (  
  card_id STRING,  
  member_id STRING,  
  member_joining_dt STRING,  
  card_purchase_dt STRING,  
  country STRING,  
  city STRING  
)  
ROW FORMAT DELIMITED  
FIELDS TERMINATED BY ','  
LOCATION '/user/hadoop/RDS/card_member';  
  
CREATE EXTERNAL TABLE IF NOT EXISTS member_score (  
  member_id STRING,  
  score INT
```

```

)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
LOCATION '/user/hadoop/RDS/member_score';

LOAD DATA INPATH
'hdfs:/user/hadoop/sqoop/import/cred_financials_data/card_member'
OVERWRITE INTO TABLE card_member;
LOAD DATA INPATH
'hdfs:/user/hadoop/sqoop/import/cred_financials_data/member_score'
OVERWRITE INTO TABLE member_score;


Insert into look_up_table select ltt.card_id,
cm.card_purchase_dt, ltt.transaction_dt, ltt.member_id,
member_joining_dt, country, city, UCL, ltt.postcode, score
from last_ten_transactions ltt
inner join
member_score ms
on ltt.member_id=ms.member_id and ltt.ranking=1
inner join
card_member cm
on cm.member_id=ltt.member_id and ltt.ranking=1
inner join
(select card_id, avg(amount)+ (3* stddev(amount)) as UCL
from last_ten_transactions
where ranking<=10
group by card_id
) as ucl
on ltt.card_id=ucl.card_id ;

exit;

```

6) Validate the data in Hbase, Hive by using below commands.

```

Hive:
    show tables
    select count(*) from card_member;
    select count(*) from member_score;
Hbase:
    List
    count 'look_up_table'

```

scan 'look_up_table'

End of task 4 we can see Table Creation and Batch Processing is completed i.e. Historical and lookup tables are successfully loaded with all the data from RDS and local file.